UNIDAD 7 · DEL CRUD AL E-COMMERCE · CLASE 3

Los productos en la página

Conectamos el front con la API usando fetch: el catálogo, el filtro y el carrito

fetch JSON DOM

Diseño Backend · 5.º año · Prof. Julia Panei

DÓNDE ESTAMOS

Falta la última punta: el navegador



Cada pieza habla solo con la de al lado. El front le pide todo a tu API y la API se lo pide a Supabase. El front no sabe que Supabase existe.

UNA PREGUNTA ANTES

¿Por qué no ir directo a Supabase?

! Si el front fuera directo

Necesitaría la clave secreta adentro de app.js. Cualquiera la ve con F12 y puede borrar tus productos o cambiar precios.

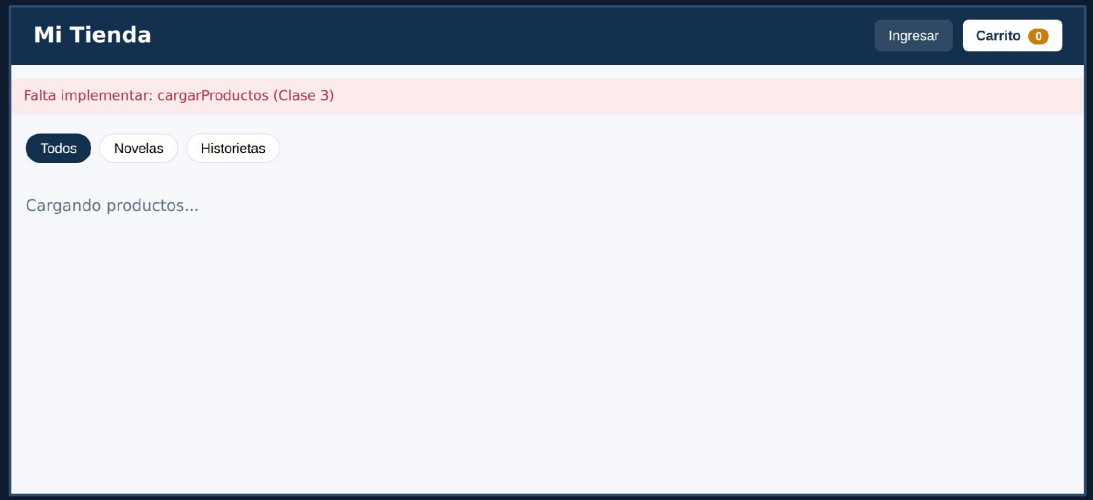
✓ Como lo hacemos

La clave vive en el .env del servidor y nunca sale de ahí. El front solo conoce rutas: /productos, /categorias.

`Todo lo que está en public/` lo puede leer cualquiera. Lo secreto va siempre del lado del servidor.

PASO 1

Así arranca la página hoy



1. **`npm start`** en la carpeta del proyecto
2. **`localhost:3000`** nunca index.html con doble clic
3. **`categorías: ya andan`** cargarCategorias es tu modelo

El aviso rojo es el TODO: lo vamos a reemplazar por el fetch que trae los productos.

LA HERRAMIENTA DE HOY

fetch en tres pasos

1	**Pedir**	`const res = await fetch('/productos');`	manda un GET a tu API
2	**Revisar**	`if (!res.ok) { ... }`	true con 200 a 299; false con 400, 500 o 501
3	**Leer**	`const productos = await res.json();`	el texto JSON pasa a ser un array

Como un delivery: fetch es hacer el pedido, res es la caja que llega, res.ok es la etiqueta y res.json() es abrirla. Sin los dos await, en vez de datos tenés una promesa.

PASO 2

cargarProductos: el hueco de hoy

```
async function cargarProductos(categoriaId) {
  categoriaActual = categoriaId || null;
  let url = '/productos';
  if (categoriaId) url += '?categoria=' + categoriaId;

  const res = await fetch(url);
  if (!res.ok) return mostrarMensaje(await res.text(),
'error');

  const productos = await res.json();
  pintarProductos(productos);     // ya resuelta
  pintarCategoriaActiva();        // ya resuelta
}
```

`'?categoria=' + id`

arma /productos?categoria=2: del otro lado es req.query

`if (!res.ok)`

si la API dio error, muestra su mensaje y corta

`res.text()`

el error llega como texto, no como JSON

PASO 3

Lo que pasa con un clic en "Novelas"



front · hoy

backend · Clase 2

Probalo: cada categoría muestra solo sus productos y su botón queda marcado. Todos los trae de vuelta.

DEL JSON A LA TARJETA

Cada dato tiene su lugar

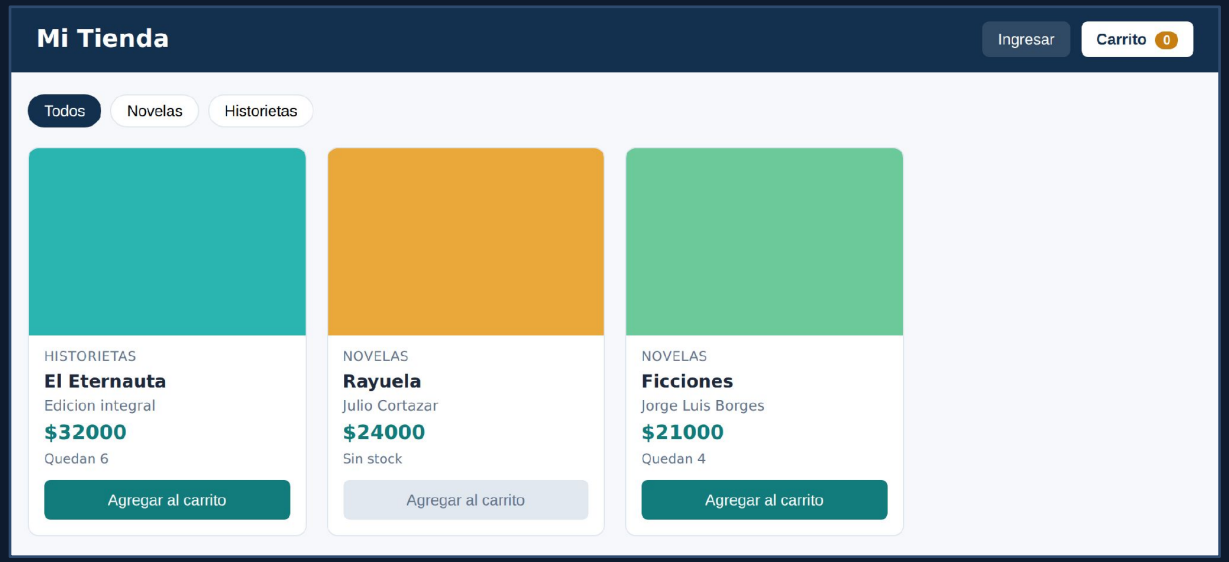
```
{
  "id": 1,
  "nombre": "El Eternauta",
  "precio": 32000,
  "stock": 6,
  "imagen_url": "https://...",
  "categorias": {
    "nombre": "Historietas"
  }
}
```

`p.imagen_url`	el src de la imagen
`p.categorias.nombre`	la etiqueta de arriba
`p.nombre, p.precio`	el título y el precio
`p.stock`	"Quedan 6", o "Sin stock" y botón apagado
`p.id`	onclick="agregar(1)" del botón

categorias.nombre llega por el categorias(nombre) del select de la Clase 2.

LA META

Así queda al final de hoy



Tarjetas
foto, categoría, precio y stock

Filtro
el botón elegido queda marcado

Sin stock
botón desactivado

Las imágenes salen de la URL pública de Storage que guardaron en imagen_url.

PASO 4

F12: mirar adentro

1 Red
Network

Filtrá Fetch/XHR y recargá: aparecen categorias y productos, los dos en 200. Tocá una categoría y aparece productos?categoria=2.

2 Respuesta
Response

Clic en productos → Respuesta: es el mismo JSON que veías en Thunder Client.

3 Consola
Console

Los errores de app.js salen en rojo, con el número de línea donde se cortó.

F12 es tu Thunder Client en el navegador: ahí ves qué pidió el front y qué le respondió la API.

PASO 5

El carrito vive en el navegador

F12 → Aplicación → Local Storage → carrito

```
[
  { "producto_id": 1, "cantidad": 2 },
  { "producto_id": 3, "cantidad": 1 }
]
```

Recargá la página: el carrito sigue ahí. Hasta que se compra, no toca la base.

Solo ids y cantidades

nunca precios: el precio se podría cambiar con F12

"Total estimado"

es para mostrar; el real lo calcula crearPedido con los precios de la base

Comprar todavía dice "Falta implementar": lo conectamos la próxima clase, junto con el login.

PASO 6

Hacela tuya

1 Nombre

`public/index.html`

el <title> y el <h1>

2 Colores

`public/estilos.css`

--principal, --acento y --destacado

3 Productos

`Supabase`

Table Editor y bucket productos

Mínimo para hoy: 6 productos en 2 categorías, todos con imagen. Si cambiás algo en Supabase, no tocás código: recargás y aparece.

OJO CON ESTO

Los errores que más van a aparecer

! No carga ni las categorías

Abriste index.html con doble clic (file://).

✓ Entrá por localhost:3000

! Sigue "Falta implementar"

No guardaste o el navegador usa la versión vieja.

✓ Guardá y Ctrl + F5

! Imagen rota

imagen_url vacío o el bucket no es público.

✓ Abrí la URL en otra pestaña

! forEach is not a function

Falta el await antes de res.json().

✓ await res.json()

EXTENSIÓN OPCIONAL

Un buscador sin ruta nueva

```
<input id="buscador" type="search"
       oninput="buscar(this.value)">
```

```
let ultimaLista = [];   // en ESTADO
// en cargarProductos, despues del res.json():
ultimaLista = productos;

function buscar(texto) {
  const t = texto.toLowerCase();
  pintarProductos(ultimaLista.filter(
    p => p.nombre.toLowerCase().includes(t)));
}
```

oninput

se llama con cada letra que escribís o borrás

toLowerCase()

"rayuela" encuentra "Rayuela"

filter

se queda con los que incluyen lo escrito

ANTES DE IRSE

El checklist de hoy

- ✓ **Catálogo** — productos con imagen, categoría, precio y stock
- ✓ **Filtro** — solo esa categoría y el botón marcado
- ✓ **Sin stock** — muestra "Sin stock" y el botón apagado
- ✓ **F12 → Red** — productos responde 200
- ✓ **Carrito** — suma, resta y sigue al recargar
- ✓ **Tu rubro** — nombre, colores y 6 productos propios

Ingresar y Comprar siguen diciendo "Falta implementar (Clase 4)". Está bien: es lo que viene.

PARA LLEVARSE

Tres ideas de hoy

1. **El front no conoce a Supabase.**
   Le pide todo a tu API; la clave secreta no sale del servidor.

2. **fetch: pedir, revisar, leer.**
   await fetch, res.ok y await res.json(), siempre en ese orden.

3. **El carrito guarda ids, no precios.**
   El precio y el total de verdad los pone el backend.

Próxima clase: registro, login y compra desde la página. La tienda queda completa de punta a punta.